

KneeVision: A Comprehensive Assistant for Knee Osteoarthritis Diagnosis

Daniel Fox, Manohar Killamsetti, Siming Li, Sarvesh Shah, Swatej Mella, Deep Patel

Seidenberg School of Computer Science and Information Systems
Pace University, New York, NY, USA

Abstract—Knee Osteoarthritis (KOA), is one of the most common causes of chronic disabilities worldwide, yet obtaining a quick and objective diagnosis for KOA is difficult because the process of manually interpreting radiographs is highly subjective. This paper introduces KneeVision; a web based program that uses Deep Learning to automatically detect the severity of knee osteoarthritis through the use of image processing technology. The development of Knee Vision utilized an aggregated database consisting of 75,000 X-ray images of knees in varying degrees of Kellgren Lawrence (KL) grade levels. These were created by combining real radiographic images with synthetic images produced via class-conditional Deep Convolutional Generative Adversarial Networks (Deep DCGANs). EfficientNet B4, which was pre-trained on ImageNet, was used as the base model, with a custom classifier added to it. In addition, this was trained utilizing a two stage fine-tuning strategy. It achieved roughly 93

Index Terms—Deep Learning, Knee Osteoarthritis, Kellgren-Lawrence Grading, Medical Imaging, X-ray Analysis, Web-based Application, Computer-Aided Diagnosis, Musculoskeletal Health, Automated Grading, EfficientNet, DCGAN.

I. INTRODUCTION

The prevalence of Knee Osteoarthritis (KOA) among older adults has increased by nearly 100% over the past few decades. KOA develops from chronic inflammation in the knee joint caused by repetitive use of the knee, resulting in gradual degradation of knee cartilage, a protective layer surrounding the bone — which ultimately leads to permanent damage to the knee structure [1]. Consequently, it has become one of the most common causes of pain and physical impairment worldwide [2] placing a substantial burden on both patients and healthcare systems. Clinical examination combined with X-rays is the primary method used to diagnose KOA; however, the Kellgren-Lawrence (KL) Grading System is the standard measurement tool for assessing the degree of knee osteoarthritis severity [3]. The KL Grade consists of five levels — Levels 0–4 — that are determined by evaluating radiographic characteristics including: joint-space loss, osteophyte formation and subchondral sclerosis. Unfortunately, determining whether each radiographic feature represents KOA is subjective. In addition to difficulty correlating the degree of symptoms reported by patients with radiographic findings, there is also a lack of consistency regarding how to grade the same X-rays by clinicians. These inconsistencies have serious implications for patients who may experience delayed treatment or incor-

rect intervention because their KOA was misgraded or were unaware of changes in their disease [4].

Therefore, there is a need for objective automated tools that assist clinicians in grading KOA severity accurately and consistently. Convolutional Neural Networks (CNN's), which are part of Deep Learning, have demonstrated potential as an effective way for automated medical image analysis — particularly for classification tasks in radiology [5]; however, applying CNN's to automate KOA grading faces unique challenges. Visual differences between adjacent KL Grades are very minor. Additionally, the limited number of publicly available large-scale labeled knee X-ray databases makes developing and training reliable models challenging. KneeVision attempts to address these issues using an end-to-end automated KOA grading system. This includes synthetic data generated using DCGANs, combined with a fine-tuned EfficientNet-B4 backbone — all designed to provide a reliable and accurate automated KOA grading system.

II. LITERATURE REVIEW

Prior to the advent of CNNs, researchers used manual methods such as Joint Space Narrowing (JSN) and the presence or absence of osteophytes. These methods were limited in their ability to be robust when dealing with variability in exposure and orientation [4].

Researchers then turned to CNNs for KOA grading, which allowed them to produce results that compared favorably to those produced by expert radiologists. Tiulpin et al. developed a Siamese CNN architecture designed to capture bilateral symmetry in the knee joint, emphasizing the importance of localized feature extraction [8]. Meanwhile, Chen et al. demonstrated the effectiveness of fine-tuning pre-trained networks such as VGG and ResNet to address subtle morphologic changes across KL grades; however, they also identified challenges associated with class imbalances in public datasets [6].

More recently, researchers have applied GANs to KOA grading to solve issues related to data scarcity. In particular, GANs generate realistic synthetic radiographs, providing a sufficient amount of training data to mitigate potential overfitting problems associated with complex backbones like EfficientNet. This paper builds upon existing research using DCGAN-based synthetic generation combined with an EfficientNet-B4 backbone to achieve a balance between computational efficiency and multi-class classification performance.

III. METHODOLOGY

A. Dataset

The development of KneeVision has been based on the use of two data sets: the Mendeley I data set (Chen, Pingjun [6]) and the Kaggle KOA data set [7] that each include images of knees from an x ray perspective covering all five Kellgren-Lawrence grades (KL grades; 0 – 4). The Mendeley I data set had 8,260 images and were in a uniform resolution of 224×224 pixels whereas the Kaggle data set had 400 images and their dimensions varied. A python script was developed to perform multiple signal Region of Interest (ROI) cropping using a joint detection method. This allowed for standardization of the ROI of the Kaggle images so they could be centered consistent with the fixed resolutions used by the Mendeley I images. Once these steps had been completed and the two data sets had been merged into one data set, there existed a total of 8,684 usable images which would be less than what would be needed to train a robust deep learning model. To address this issue, five class conditional DCGAN's (Deep Convolutional Generative Adversarial Networks), one per KL grade were created and trained on the complete, preprocessed, combined real data set. These models generated synthetic x-ray images that when added to the original data set produced a final training data set that included 75,000 images with 15,000 images per KL grade.

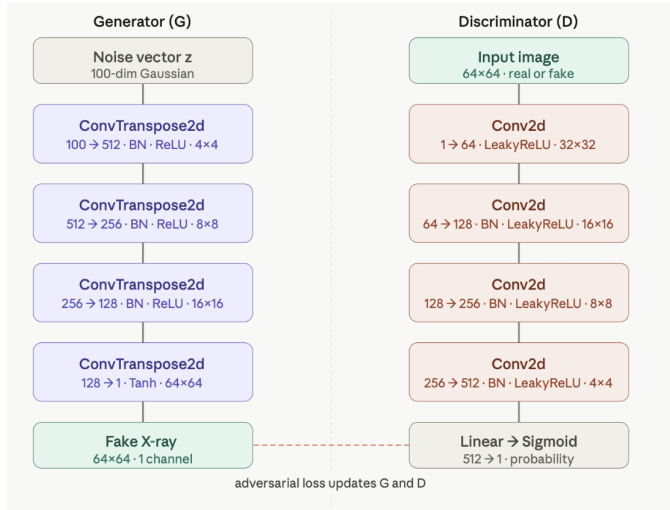


Fig. 1: DCGAN architecture used for synthetic X-ray generation per KL grade.

B. Frontend Architecture

The Frontend for KneeVision has been developed using Typescript and React 18. Vite is used as our Build Environment and allows us to reload the frontend (Hot Reload) while developing. It also generates the frontend quickly with Tree Shaking when we are building for Production. Tailwind CSS has provided us a Utility-First Styling Framework that uses Mobile First Breakpoints. Additionally it supports Dark Mode out of the box. Our Application consists of Five Pages:

Landing Page, Login Page, Registration Page, Upload Page, Image Gallery Page. All State Management within the Application has been done by Using React's Hooks `useState` and `useEffect`. We have also configured Vite to forward client side requests to our Express Backend via Proxy, thus avoiding CORS Violation. We have Implemented file upload Functionality via the `react-dropzone` Library, which Supports the Following Formats: JPG, PNG, WebP, DICOM. Additionally it includes Live Preview and Visualization of Upload Progress. Our Interface was Prototyped Initially in Figma. In addition, the Landing Page reflects the Platform's Target Metrics. These include: over 95% Model Accuracy, less than 5 Seconds Analysis Time, Full Kl Grade 0 – 4 Coverage.

C. Backend Architecture

Backend functionality is written in Node.js utilizing an Express.js non-blocking I/O web development structure. The non-blocking I/O development environment was selected due to its ability to support simultaneous processes (e.g., uploading images and performing inference) and its scalability. Three different types of APIs exist to be used within the backend application: Authentication Endpoints (`/api/v1/auth/register`, `/api/v1/auth/login`) Image Handling End Points (`/uploads/presign`, `/uploads/:key`, `/api/v1/images`) Health Checks (`/health`) To authenticate without state (stateless) JSON Web Tokens (JWT) will be issued after a successful login. Each token contains two claims, `userId` and `userEmail`. Once the token has expired after one hour it will no longer work. A middleware called `requireAuth` exists to verify that the token provided by the client still exists, has not expired, and/or does not have any errors. If there are any of these issues then a HTTP error code of 401 will be sent back to the client. Password Hashing will utilize `bcrypt` with ten rounds. Sharp library provides server side image processing. It will take an image uploaded from the client, convert the image to grayscale and resizes the image while preserving the aspect ratio so that when the resized image is passed through our models they can process it.

D. Database and Cloud Infrastructure

For the backend of KneeVision, MongoDB is used as its main database and it's being hosted through MongoDB Atlas. Three collections exist in this database — users (`userId`, email, passwordHash, and time-stamps; there exists a unique index for 'email' so that no two identical emails can be created), images (`fileUrl`, `originalName`, `contentType`, `uploadedBy` and time-stamp) and prediction (`imageId`, KL grade, confidence level of the model, each class probabilities, and version of model); the schema allows for many different models and predictions to be generated from one image. When we move into production we are going to use AWS to host our application — we will store all our images in Amazon S3, our server running express.js and our ML inference engine running on either an EC2 instance or lambda, and we will deliver all our frontend assets using cloudfront cdn. In sprint one we

stored images locally on a mounted filesystem at /uploads/ along with returning presigned endpoints containing a path to the disk where they are located.

E. DevOps and Quality Assurance

Version Control and Collaboration All project management, including version control and collaboration, is handled by GitHub. The method used to protect the main branch from being directly committed to it is the use of a branch protection strategy. Each feature is built in its own independent feature branch, which then is pushed into the repository as part of a pull request. The pull request requires approval from at least one other developer before changes can be accepted. In addition, there is a continuous integration/continuous deployment (CI/CD) pipeline that executes all of the testing scripts each time code is pushed into the repository. The project backlog and sprint planning are also executed within Jira. Commits messages link to the related Jira SCRUM ticket. ESLint and Prettier enforce code quality standards for each module, and are run against all code prior to committing it to the repository. This is done using Husky's pre-commit hook functionality. Jest is used to execute both backend unit and integration testing. It is also used for frontend unit testing. Vitest is used to execute frontend component testing using React Testing Library. For example, during Sprint 1, a total of 86 tests were written across 8 modules resulting in zero failed tests and an estimated coverage rate of approximately 89% based upon the number of lines of code tested. Backend modules include: `imageProcessor.js` (11 tests, ~95%), `db.js` (6 tests, ~90%), and API Routes (17 tests, ~88%). Frontend modules include: `LoginForm` (10 tests, ~92%), `RegisterForm` (10 tests, ~91%), `ImageGallery` (8 tests, ~90%), `ImageUploader` (8 tests, ~85%), and `Pages` (16 tests, ~88%).

F. Machine Learning Pipeline

1) *Data Augmentation via DCGAN*: A variety of methods can be used to perform data augmentation. In this study we will use deep generative models. We will train five different DCGANs; one DCGAN for every KL-grade. Each DCGAN is composed of a generator and a discriminator. The generator is a neural network whose input is a 100-dimensional gaussian noise vector and generates, by progressive upsampling via four transposed convolution layers followed by ReLU activation functions and batch normalisation, a synthetic 64x64 x-ray image. The discriminator also has four layers but instead of generating synthetic images it uses four stride convolutional layers followed by Leaky ReLU activation functions and batch normalisation to generate a scalar value that represents the discriminator's confidence that the synthetic image is real or not. These two networks have been trained against each other in order to make the discriminator distinguish between real and synthetic images. As the discriminator improves the generator will improve because the discriminator will find it harder to tell if an image is real or not. After all these generators had been trained we generated synthetic images from them and added those to our real dataset to get a total training dataset of

75,000 images, with 15,000 images per KL-grade. Initially we attempted to classify the X-rays using a compact transformer based model called CCT. We found that CCT could achieve an accuracy of about 57% on the validation set but did not show much improvement after increasing the number of training iterations. One reason for this lack of improvement may have been the fact that the CCT model relies heavily on self-attentions to identify important features within an image. Self-attentions require large amounts of training data so that they can reliably identify useful relationships between features. Even though we trained on 75,000 images, the self-attentions in the CCT model were unable to generalise well enough. Therefore we decided to use a traditional CNN based model, specifically EfficientNet-B4. Unlike transformers, CNNs have many built-in biases towards what they expect to see. For example, they assume that features close together are related and that the position of objects does not matter. As a result, CNNs are very good at identifying structured patterns in images. Knee X-rays contain some of these patterns, including joint space narrowing, osteophytes and subchondral sclerosis. The EfficientNet-B4 model increases the depth, width and spatial resolution of the network simultaneously using a single hyperparameter known as a compound coefficient. Because of this, EfficientNet-B4 has significantly more representation power than smaller versions of itself while still having less computational overhead. EfficientNet-B4 also comes pre-trained on ImageNet so it already contains many rich low-level representations that can be transferred into the radiography domain. The base model we chose to fine-tune consisted of the original EfficientNet-B4 classifier replaced by a new custom classification layer. This custom classification layer consists of `dropout(0.4)` -> `linear(1792, 512)` -> `relu` -> `dropout(0.3)` -> `linear(512, 5)`. This maps the 1792 dimensional pooled feature vector produced by the EfficientNet-B4 model onto the five classes in the KL grading system. The size of the total model is approximately 17.5 million parameters. The model architecture is depicted in figure 1.

Fine-tuning was performed in two stages. Firstly, we froze the backbone and only trained the custom classification head for 15 epochs using Adam optimiser with initial learning rate = 0.001 and ReduceLROnPlateau scheduling. By doing this we ensured that when we start training the backbone we don't immediately overwrite its pre-training features. At the same time we established a stable representation in the classification head which reached a maximum validation accuracy of 60.57%. Secondly, we un-froze both the backbone and the custom classification head and continued to train until convergence was reached or for a maximum of 150 epochs whichever came first. During this second stage we differentiated the learning rate for the backbone and the classification head i.e. $3e-5$ for the backbone and $1.5e-4$ for the classification head. Using this technique we avoided over-writing the pre-trained features in the backbone while at the same time enabling it to adjust to the characteristics of the radiographic domain. To prevent over-confidence in prediction we used label-smoothing with $\epsilon=0.1$. To add robustness to the

KneeVision — EfficientNet-B4 Training Process

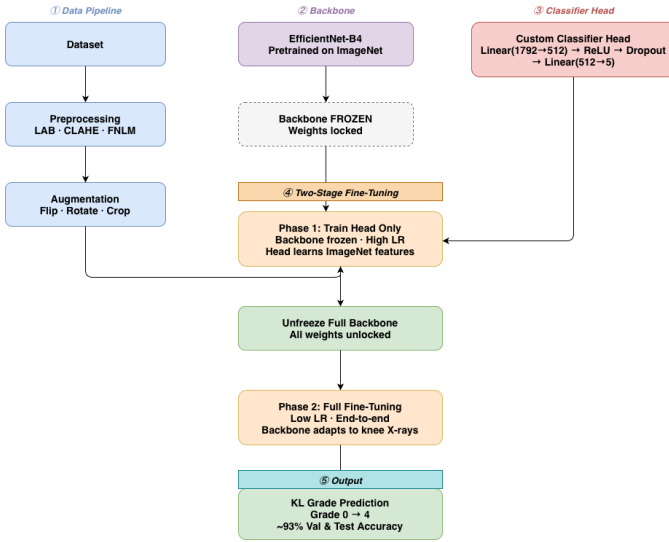


Fig. 2: EfficientNet-B4 model architecture with custom classification head.

model we used mix-up-augmentation with $\alpha=0.2$ (i.e., blend pairs of training images and their corresponding labels each epoch). To augment the model online during training we randomly cropped (from 256 to 224), horizontally flipped, rotated (by ± 10 degrees), translated (by 5%) affinely scaled (between 95% and 105%) colour jittered (brightness/contrast $\pm 20\%$), and randomly erased (probability $p = 0.2$). We used early-stopping with patience = 30 epochs to stop training when performance on the validation set stopped improving.

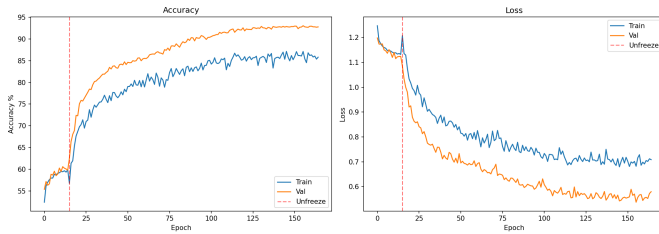


Fig. 3: Training Curves.

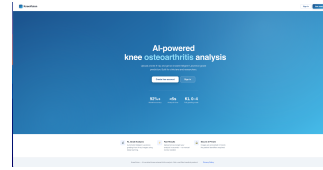
IV. FUTURE ENHANCEMENTS

Although KneeVision has demonstrated superior performance, there are many ways to update it so that it can transform from a student's project into an application ready for use in clinics:

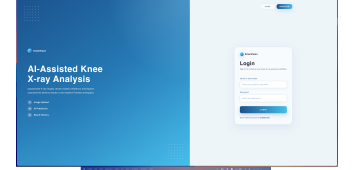
- **Mobile Inference Using TensorFlow Lite or ONNX Runtime:** The model will be compressed to allow local inference on mobile devices, enabling physical therapists to receive immediate diagnostic information within the exam room without needing a high-speed network connection.

- **Detection of Surgical Hardware (e.g. screws/prosthetics):** Future versions will include a YOLOv8 pre-processing layer. This will assist the KL-grader in identifying screws or other prosthetic hardware rather than being confused by metal artifacts, increasing the reliability of assessments for post-surgical patients.

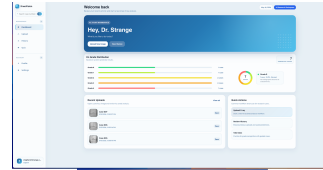
V. SCREENSHOTS



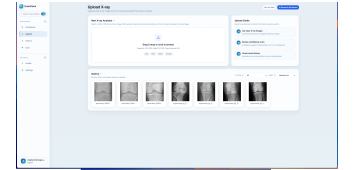
(a) Landing Page



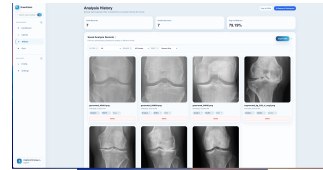
(b) Register/Login Page



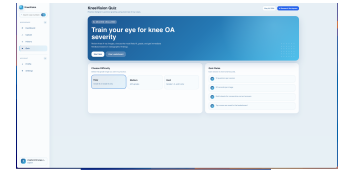
(a) Dashboard (Light Mode)



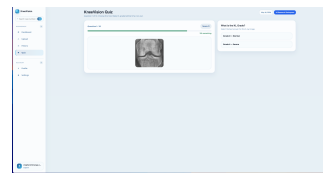
(b) Upload Page



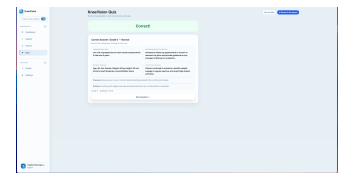
(a) History Page



(b) Quiz Page

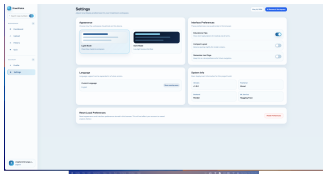


(a) Quiz Leaderboard



(b) Profile Settings

Fig. 7: Quiz Interface



(a) Website Settings



(b) Results Page

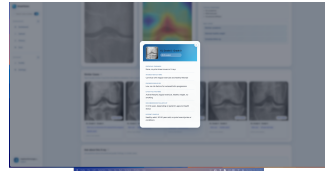
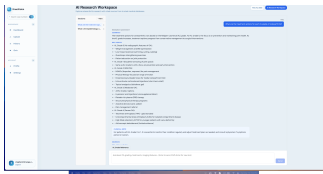
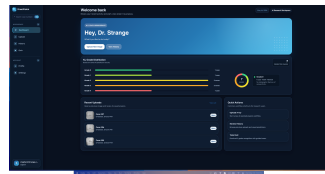


Fig. 10: Results Page



(a) Research Module



(b) Dashboard (Dark Mode)

VI. CONCLUSION

This research introduced KneeVision; a complete deep learning based approach that includes both a machine learning model/web application to standardize the assessment of knee osteoarthritis. We utilized class-conditionally trained Deep Generative Adversarial Networks (DCGAN) to increase the size of our dataset to 75,000 images. With the use of a two stage fine tuning method with an EfficientNet B4 backbone, the KneeVision has been able to reach an accuracy of 93% at providing consistent results. As such, KneeVision can be viewed as a cost-effective prototype for computer assisted diagnosis and it is capable of reducing observer bias while enhancing patient care in musculoskeletal medicine

REFERENCES

- [1] R. C. Lawrence et al., "Estimates of the prevalence of arthritis and other rheumatic conditions in the United States," *Arthritis & Rheumatism*, vol. 58, no. 1, pp. 26–35, 2008.
- [2] D. T. Felson, "Epidemiology of hip and knee osteoarthritis," *Epidemiologic Reviews*, vol. 10, pp. 1–28, 1988.
- [3] J. H. Kellgren and J. S. Lawrence, "Radiological assessment of osteoarthritis," *Annals of the Rheumatic Diseases*, vol. 16, no. 4, pp. 494–502, 1957.
- [4] A. Kinds et al., "Radiographic features of osteoarthritis and inter-observer variability," *Osteoarthritis and Cartilage*, vol. 20, no. 9, pp. 981–986, 2012.
- [5] G. Litjens et al., "A survey on deep learning in medical image analysis," *Medical Image Analysis*, vol. 42, pp. 60–88, 2017.
- [6] P. Chen, "Knee osteoarthritis severity grading dataset," Mendeley Data, 2018. [Online]. Available: <https://data.mendeley.com/datasets/56rmx5bjcr/1>
- [7] "Knee osteoarthritis dataset with severity grading," Kaggle. [Online]. Available: <https://www.kaggle.com/datasets/shashwatwork/knee-osteoarthritis-dataset-with-severity>
- [8] A. Tulpin et al., "Automatic Knee Osteoarthritis Diagnosis from Plain Radiographs: A Deep Learning-Based Approach," *Scientific Reports*, vol. 8, no. 1, p. 1727, 2018.